

ID	Question	Scenario	Scenario description	Domain	Task statement	Question type/ pattern	Answer options	Correct Answer	Reasoning	Wrong answer	Reasoning
1	What is the primary technical constraint when considering batch processing for this workflow?	Claude Code for Continuous Integration	Claude Code for Continuous Integration You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.				A The batch API doesn't support tool definitions in request parameters.	C	Correct! This is correct. The batch API's asynchronous fire-and-forget model means there is no mechanism to intercept a tool call mid-request, execute the tool, and return results for Claude to continue its analysis. This fundamentally breaks iterative tool-calling workflows that require multiple rounds of tool invocation and response within a single logical interaction.	A	The batch API doesn't support tool definitions in request parameters.
							B Batch processing latency of up to 24 hours is too slow for pull request feedback, though the workflow could otherwise function.				
							C The asynchronous model prevents executing tools mid-request and returning results for Claude to continue analysis.				
2	Your CI/CD system performs three types of Claude-powered analysis: (1) quick style checks on each PR that block merging until complete, (2) comprehensive security audits of the entire codebase run weekly, and (3) test case generation triggered nightly for recently-modified modules. The Message Batches API offers 50% cost savings but can take up to 24 hours to process. You want to optimize API costs while maintaining acceptable developer experience. Which combination correctly matches each task to its API approach?	Claude Code for Continuous Integration	Claude Code for Continuous Integration You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.				D Batch processing lacks request correlation identifiers for matching outputs to input requests.	C	The asynchronous model prevents executing tools mid-request and returning results for Claude to continue analysis.		B Batch processing latency of up to 24 hours is too slow for pull request feedback, though the workflow could otherwise function.
							A Use synchronous calls for PR style checks; use the Message Batches API for weekly security audits and nightly test generation.				
							B Use synchronous calls for PR style checks and nightly test generation; use Message Batches API only for weekly security audits.				
3	Your automated code review averages 15 findings per pull request, with developers reporting a 40% false positive rate. The bottleneck is investigation time: developers must click into each finding to read Claude's reasoning before deciding whether to address or dismiss it. Your CLAUDE.md already contains comprehensive rules for acceptable patterns, and stakeholders have rejected any approach that filters findings before developer review. What change would best address the investigation time bottleneck?	Claude Code for Continuous Integration	Claude Code for Continuous Integration You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.				C Use synchronous calls for all three tasks for consistent response times, and rely on prompt caching to reduce costs across all workloads.	A	Correct! This is the correct approach. PR style checks block developers and require immediate responses via synchronous calls, while weekly security audits and nightly test generation are scheduled tasks with flexible timelines that can easily tolerate the up-to-24-hour batch processing window, capturing the 50% cost savings on both.		
							D Use the Message Batches API for all three tasks to maximize the 50% cost savings, and configure the pipeline to poll for batch completion.				
							A Categorize findings as "blocking issues" versus "suggestions" with tiered review requirements				
							B Configure Claude to only surface findings it assesses as high confidence, filtering out uncertain flags before developers see them	C	Correct! Including reasoning and confidence assessments inline with each finding directly addresses the investigation time bottleneck by allowing developers to quickly evaluate findings without clicking into each one separately. This approach respects the constraint against filtering, since all findings remain visible while making triage significantly faster.		
							C Require Claude to include its reasoning and confidence assessment inline with each finding				
							D Add a post-processor that analyzes finding patterns and automatically suppresses those matching historical false positive signatures				

Your CI pipeline includes two Claude-powered code review modes: a pre-merge-commit hook that blocks PR merging until complete, and "deep analysis" that runs overnight, polls for batch completion, then posts detailed suggestions to the PR. You want to reduce API costs using the Message Batches API, which offers 50% cost savings but requires polling and may take up to 24 hours to complete. Which mode should use batch processing?

Claude Code for Continuous Integration

Claude Code for Continuous Integration
You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

Your team uses Claude Code to generate code suggestions, but you notice a pattern: subtle issues—performance optimizations that break edge cases, cleanups that change behavior unexpectedly—only surface when a different team member reviews the PR. Claude's reasoning during generation shows it considered these cases but concluded its approach was correct. Which approach directly addresses the root cause of this self-review limitation?

Claude Code for Continuous Integration

Claude Code for Continuous Integration
You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

Your pipeline script runs `claude "Analyze this pull request for security issues"` but the job hangs indefinitely. Logs indicate Claude Code is waiting for interactive input. What's the correct approach to run Claude Code in an automated pipeline?

Claude Code for Continuous Integration

Claude Code for Continuous Integration
You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

A
Neither mode

B
Deep analysis only

C
Pre-merge-commit hook only

D
Both modes

B
Deep analysis only

Correct!
Deep analysis is the ideal candidate for batch processing because it already runs overnight, tolerates latency, and uses a polling model to check for completion before posting results—perfectly matching the Message Batches API's asynchronous, poll-based design while capturing the 50% cost savings.

A
Enable extended thinking mode for the generation pass, allowing more thorough deliberation before producing suggestions.

B
Add explicit self-review instructions to the generation prompt, asking Claude to critique its own suggestions before finalizing output.

C
Include comprehensive test files and documentation in the prompt context so Claude better understands expected behavior during generation.

D
Have a second, independent Claude Code instance review the changes without seeing the generator's reasoning.

D
Have a second, independent Claude Code instance review the changes without seeing the generator's reasoning.

Correct!
Using a second, independent Claude Code instance without access to the generator's reasoning directly addresses the root cause by eliminating confirmation bias. This fresh perspective mirrors the benefit of human peer review, where a different team member catches issues the original author rationalized away.

A
Add the `-p` flag: `claude -p "Analyze this pull request for security issues"`

B
Redirect stdin from `/dev/null`: `claude "Analyze this pull request for security issues" < /dev/null`

C
Set the environment variable `CLAUDE_HEADLESS=true` before running the command

D
Add the `--batch` flag: `claude --batch "Analyze this pull request for security issues"`

A
Add the `-p` flag: `claude -p "Analyze this pull request for security issues"`

Correct!
The `-p` (or `--print`) flag is the documented way to run Claude Code in non-interactive mode. It processes the given prompt, outputs the result to stdout, and exits without waiting for user input, making it ideal for CI/CD pipelines.

A pull request modifies 14 files across the stock tracking module. Your single-pass review analyzing all files together produces inconsistent results: detailed feedback for some files but superficial comments for others, obvious bugs missed, and contradictory feedback—flagging a pattern as problematic in one file while approving identical code elsewhere in the same PR. How should you restructure the review?

Claude Code for Continuous Integration

Claude Code for Continuous Integration
You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

Analysis of your automated code review shows significant variation in false positive rates across finding categories. Security and correctness findings have an 8% false positive rate, performance findings have 18%, style and naming findings have 52%, and documentation findings have 48%.

Developer surveys indicate growing distrust—many have started dismissing findings without review because “half are wrong.” The high false positive categories are undermining confidence in the accurate categories. What approach best restores developer trust while improving the system?

Claude Code for Continuous Integration

Claude Code for Continuous Integration
You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

A
Split into focused passes: analyze each file individually for local issues, then run a separate integration-focused pass examining cross-file data flow.

B
Run three independent review passes on the full PR and only flag issues that appear in at least two of the three runs.

C
Require developers to split large PRs into smaller submissions of 3-4 files before the automated review runs.

D
Switch to a higher-tier model with a larger context window to give all 14 files adequate attention in one pass.

A
Keep all categories enabled while adding few-shot examples to improve each category's accuracy over the coming weeks.

B
Keep all categories but display a confidence score with each finding, letting developers decide which to investigate.

C
Temporarily disable high false positive categories (style, naming, documentation) and run only high-precision categories while improving prompts.

D
Apply a uniform strictness reduction across all categories to bring the overall false positive rate to an acceptable level.

Correct!
Splitting the review into focused per-file passes directly addresses the root cause of attention dilution, ensuring consistent depth and catching local issues reliably. A separate integration-focused pass then handles cross-file concerns like data flow dependencies, covering both dimensions of review quality.

A
Split into focused passes: analyze each file individually for local issues, then run a separate integration-focused pass examining cross-file data flow.

C
Temporarily disable high false positive categories (style, naming, documentation) and run only high-precision categories while improving prompts.

Correct!
Temporarily disabling the high false positive categories (style, naming, documentation) immediately stops trust erosion by removing the noise that causes developers to dismiss all findings, while preserving the value of high-precision categories like security and correctness. This approach allows time to improve prompts for the problematic categories before re-enabling them, rebuilding trust through demonstrated accuracy.

Your automated reviews identify valid issues but developers report the feedback isn't actionable. Findings say things like "complex ticket allocation logic" or "potential null pointer" without specifying what to change. When you add detailed instructions like "always include specific fix suggestions," the model still produces inconsistent output—sometimes detailed, sometimes vague. What prompting technique would most reliably produce consistently actionable feedback?

Claude Code for Continuous Integration

Claude Code for Continuous Integration
You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

Your team wants to reduce API costs for automated analysis. Currently, real-time Claude calls power two workflows: (1) a blocking pre-merge check that must complete before developers can merge, and (2) a technical debt report generated overnight for review the next morning. Your manager proposes switching both to the Message Batches API for its 50% cost savings. How should you evaluate this proposal?

Claude Code for Continuous Integration

Claude Code for Continuous Integration
You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

After an initial automated review generates 12 findings, a developer pushes new commits to address the issues. When the review runs again, it produces 8 findings—but developers report that 5 duplicate earlier comments on code that was already fixed in the new commits. What's the most effective way to eliminate this redundant feedback while maintaining thorough analysis?

Claude Code for Continuous Integration

Claude Code for Continuous Integration
You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

A

Further refine the instructions with more explicit requirements for each part of the feedback format (location, issue, severity, suggested fix)

B

Implement a two-pass approach where one prompt identifies issues and a second prompt generates fixes, allowing specialization

C

Add 3-4 few-shot examples showing the exact format you want: issue identified, code location, specific fix suggestion

D

Expand the context window to include more of the surrounding codebase so the model has sufficient information to suggest specific fixes

C

Add 3-4 few-shot examples showing the exact format you want: issue identified, code location, specific fix suggestion

Correct!

Few-shot examples are the most effective technique for achieving consistent output format when instructions alone produce variable results. Providing 3-4 examples showing the exact desired format (issue, location, specific fix) gives the model a concrete pattern to follow, which is more reliable than abstract instructions.

A

Use batch processing for the technical debt reports only; keep real-time calls for pre-merge checks.

B

Keep real-time calls for both workflows to avoid batch result ordering issues.

C

Switch both workflows to batch processing with status polling to check for completion.

D

Switch both to batch processing with a timeout fallback to real-time if batches take too long.

A

Use batch processing for the technical debt reports only; keep real-time calls for pre-merge checks.

Correct!

This is the correct approach because the Message Batches API's up to 24-hour processing time with no guaranteed latency SLA makes it ideal for overnight technical debt reports but unsuitable for blocking pre-merge checks where developers are waiting. This matches each workflow to the appropriate API based on its latency requirements.

A

Include prior review findings in context, instructing Claude to only report new or still-unaddressed issues.

B

Add a post-processing filter that removes findings matching previous file paths and issue descriptions before posting comments.

C

Run reviews only on initial PR creation and final pre-merge state, skipping intermediate commits.

D

Restrict the review scope to only files modified in the most recent push, excluding files from earlier commits.

A

Include prior review findings in context, instructing Claude to only report new or still-unaddressed issues.

Correct!

Including prior review findings in context allows Claude to intelligently distinguish between new issues and those already addressed by recent commits. This approach maintains thorough analysis while leveraging Claude's reasoning ability to avoid redundant feedback on fixed code.

		Claude Code for Continuous Integration You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.
12	Your automated code review system shows inconsistent severity ratings—similar issues like null pointer risks receive "critical" severity in some PRs but only "medium" in others. Developer trust is declining because teams can't predict which findings require immediate attention. What's the most effective way to improve severity consistency?	Claude Code for Continuous Integration

		Claude Code for Continuous Integration You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.
13	Your automated review generates test case suggestions for each PR. When reviewing a PR that adds course completion tracking, Claude suggests 10 test cases but developer feedback indicates 6 duplicate scenarios already covered in the existing test suite. What change would most effectively reduce duplicate suggestions?	Claude Code for Continuous Integration

A	Modify the prompt to ask Claude to rate severity relative to other issues in the same PR, so the most severe issue is always marked critical and others rated proportionally		
B	Request that Claude include its reasoning for each severity assignment, then use that reasoning to manually calibrate and adjust ratings during review		The correct answer is C: Including explicit severity criteria with concrete code examples directly addresses the root cause of inconsistency by removing ambiguity about what each severity level means. This is a proven prompt engineering technique that gives the model clear reference points for classification, leading to more reliable and predictable severity assignments.
C	Include explicit severity criteria in your prompt with concrete code examples for each severity level		D Add a CLAUDE.md file that lists issue types and their default severities, instructing Claude to reference this mapping when assigning ratings
D	Add a CLAUDE.md file that lists issue types and their default severities, instructing Claude to reference this mapping when assigning ratings	C Include explicit severity criteria in your prompt with concrete code examples for each severity level	A static issue-type-to-severity mapping loses important context, since the same issue type (e.g., a null pointer risk) may warrant different severities depending on factors like code path, exposure, or criticality of the affected component. This rigid approach oversimplifies severity assignment and can lead to inaccurate ratings.
A	Reduce requested suggestions from 10 to 5, assuming Claude will prioritize the most valuable cases first		
B	Include the existing test file in the context so Claude can identify what scenarios are already covered		Correct! Including the existing test file in the context directly addresses the root cause of duplication: Claude can only avoid suggesting already-covered scenarios if it knows what tests already exist. This gives Claude the information needed to reason about which suggestions would be genuinely new and valuable.
C	Add instructions directing Claude to focus exclusively on edge cases and error conditions rather than successful paths		
D	Implement post-processing that filters suggestions whose descriptions match keywords from existing test names	B Include the existing test file in the context so Claude can identify what scenarios are already covered	

Your CI pipeline runs the Claude Code CLI (with --print mode) using CLAUDE.md to provide project context for code reviews, and developers generally find the reviews insightful. However, they report that integrating findings into your workflow is difficult—Claude produces narrative paragraphs that must be manually copied into PR comments. Your team wants to automatically post each finding as a separate inline PR comment at the relevant code location, which requires structured data with file path, line number, severity, and suggested fix. What's the most effective approach?

Claude Code for Continuous Integration

Claude Code for Continuous Integration
You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

Your automated review analyzes comments and docstrings. The current prompt instructs Claude to "check that comments are accurate and up-to-date." Findings frequently flag acceptable patterns (TODO markers, straightforward descriptions) while missing comments that describe behavior the code no longer implements. What change addresses the root cause of this inconsistent analysis?

Claude Code for Continuous Integration

Claude Code for Continuous Integration
You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

A
Include explicit formatting instructions in your review prompt requiring each finding to follow a parseable template like [FILE:path] [LINE:n] [SEVERITY:level]

B
Keep the narrative review format but add a summarization step that uses Claude to generate a structured JSON summary of the findings.

C
Use CLI flags --output-format json and --json-schema to enforce structured findings, then parse output to post inline comments via the GitHub API.

D
Add a "Review Output Format" section to CLAUDE.md with examples showing structured findings, so Claude learns the expected format from project context.

Correct!
Using `--output-format json` with `--json-schema` enforces structured output at the CLI level, guaranteeing well-formed JSON with the required fields (file path, line number, severity, suggested fix) that can be reliably parsed and posted as inline PR comments via the GitHub API. This is the most effective approach because it leverages native CLI capabilities designed specifically for structured output enforcement.

A
Add few-shot examples of misleading comments to help the model recognize similar patterns in the codebase

B
Specify explicit criteria: flag comments only when their claimed behavior contradicts actual code behavior

C
Filter out TODO, FIXME, and descriptive comment patterns before analysis to reduce noise

D
Include git blame data so Claude can identify comments that predate recent code modifications

The correct answer is B:
Specifying explicit criteria—flag comments only when their claimed behavior contradicts actual code behavior—directly addresses the root cause by replacing the vague instruction with a precise definition of what constitutes a problem. This eliminates both false positives on acceptable patterns and false negatives on genuinely misleading comments.

A
Add few-shot examples of misleading comments to help the model recognize similar patterns in the codebase

While few-shot examples of misleading comments can help the model recognize similar patterns, this approach won't generalize well to novel types of contradictions because it relies on pattern-matching rather than defining the underlying criterion the model should apply.

The web search subagent returns results for only 3 of 5 requested source categories (competitor websites and industry reports succeeded, but news archives and social media feeds timed out). The document analysis subagent successfully processed all provided documents. The synthesis subagent must now produce a findings summary from this mixed-quality input. What's the most effective error propagation strategy?

Multi-Agent Research System

Multi-Agent Research System
You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

During testing, combined outputs from the web search agent (85K tokens including page content) and the document analysis agent (70K tokens including reasoning chains) total 155K tokens, but the synthesis agent performs optimally with inputs under 50K tokens. What's the most effective solution?

Multi-Agent Research System

Multi-Agent Research System
You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

A
Have the synthesis subagent request the coordinator retry the timed-out sources with extended timeouts before proceeding, ensuring complete data coverage before synthesis begins.

B
Structure the synthesis output with coverage annotations indicating which findings are well-supported versus which topic areas have gaps due to unavailable sources.

C
Have the synthesis subagent return an error to the coordinator indicating incomplete upstream data, triggering a full retry or task failure.

D
Proceed with synthesis using only the successful sources, generating output without

A
Add an intermediate summarization agent that condenses findings before passing to synthesis

B
Store findings in a vector database and give the synthesis agent retrieval tools to query during its work

C
Modify upstream agents to return structured data (key facts, citations, relevance scores) instead of verbose content and reasoning

D
Have the synthesis agent process findings in sequential batches, maintaining running state between calls

B
Structure the synthesis output with coverage annotations indicating which findings are well-supported versus which topic areas have gaps due to unavailable sources.

Correct!
Structuring the synthesis output with coverage annotations embodies graceful degradation with transparency, allowing downstream consumers and end users to understand which findings are well-supported and which topic areas have gaps. This approach preserves the value of completed work while propagating uncertainty information so informed decisions can be made about confidence levels.

The correct answer is C: Modifying upstream agents to return structured data (key facts, citations, relevance scores) addresses the root cause by reducing token volume at the source while preserving essential information. This eliminates verbose page content and reasoning chains that inflate token counts without adding value for the synthesis step.

A
Add an intermediate summarization agent that condenses findings before passing to synthesis

Adding an intermediate summarization agent introduces additional latency, cost, and risk of information loss, while the summarization agent itself must still process the full 155K tokens. This treats the symptom rather than the root cause of upstream agents producing unnecessarily verbose output.

The document analysis subagent frequently encounters failures when processing PDF files—some have corrupted sections causing parsing exceptions, others are password-protected, and occasionally the parsing library times out on large files. Currently, any exception immediately terminates the subagent and returns an error to the coordinator, which must decide whether to retry, skip the document, or fail the entire research task. This is causing excessive coordinator involvement in routine error handling. What's the most effective architectural

18 improvement?

Multi-Agent Research System

Multi-Agent Research System
You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

The document analysis subagent encounters a corrupted PDF file it cannot parse. When designing the system's error handling, what is the most effective way to handle this

19 failure?

Multi-Agent Research System

Multi-Agent Research System
You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

A
Have the coordinator validate all documents before dispatching to the subagent, rejecting documents likely to cause failures to ensure the subagent only receives processable files.

B
Configure the subagent to always return partial results with success status, embedding error details in metadata. The coordinator treats all responses as successful and filters problematic items during synthesis.

C
Create a dedicated error-handling agent that monitors all subagent failures via a shared queue and makes recovery decisions independently, dispatching retry commands directly to subagents.

D
Have the subagent implement local recovery for transient failures and only propagate errors it cannot resolve to the coordinator, including what was attempted and any partial results obtained.

Correct!
Implementing local recovery for transient failures within the subagent follows the principle of handling errors at the lowest level capable of resolving them. This reduces excessive coordinator involvement while still escalating truly unresolvable issues with full context, including what recovery was attempted and any partial results obtained.

A
Return the error with context to the coordinator agent, letting it decide how to proceed.

B
Automatically retry parsing the document three times with exponential backoff before reporting failure.

C
Throw an exception that terminates the entire research workflow.

D
Silently skip the corrupted document and continue processing other files to avoid interrupting the workflow.

A
Return the error with context to the coordinator agent, letting it decide how to proceed.

Correct!
Returning the error with context to the coordinator agent is the most effective approach because it enables the coordinator to make an informed decision—such as skipping the file, trying an alternative parsing method, or notifying the user—while maintaining visibility into the failure.

When researching a broad topic, you observe that the web search agent and document analysis agent are both investigating the same subtopics, resulting in significant overlap in their findings. Token usage has nearly doubled without proportionally increasing the breadth or depth of research coverage. What's the most effective way to address this?

Multi-Agent Research System

Multi-Agent Research System
You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

The web search and document analysis agents have both completed their tasks and returned findings to the coordinator. What is the appropriate next step for producing an integrated research output?

Multi-Agent Research System

Multi-Agent Research System
You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

A
Convert to sequential execution where document analysis runs only after web search completes, using the web search findings as context to avoid duplication

B
Allow both agents to complete their parallel work, then have the coordinator deduplicate overlapping findings before passing to the synthesis agent

C
Implement a shared state mechanism where agents log their current focus area, allowing other agents to dynamically avoid duplicating work in progress

D
Have the coordinator explicitly partition the research space before delegation, assigning distinct subtopics or source types to each agent

D
Have the coordinator explicitly partition the research space before delegation, assigning distinct subtopics or source types to each agent

Correct!
Having the coordinator explicitly partition the research space before delegation is the most effective approach because it addresses the root cause—unclear task boundaries—before any work begins. This preserves the benefits of parallel execution while preventing duplicated effort and wasted tokens.

A
Each agent directly sends its findings to the report generation agent, bypassing the coordinator

B
The document analysis agent requests the web search results and merges them internally

C
The coordinator concatenates the raw outputs from both agents and returns them as the final result

D
The coordinator passes both sets of findings to the synthesis agent for unified integration

D
The coordinator passes both sets of findings to the synthesis agent for unified integration

Correct!
This is correct because the orchestrator-workers pattern requires the coordinator to maintain centralized control by collecting results from subagents and routing them to the appropriate next component—in this case, the synthesis agent, which is specifically designed to unify and integrate findings into a coherent output.

The web search subagent times out while researching a complex topic. You need to design how this failure information flows back to the coordinator agent. Which error propagation approach best enables intelligent recovery?

Multi-Agent Research System

Multi-Agent Research System
You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

Production monitoring reveals inconsistent synthesis quality. When aggregated results total ~75K tokens, the synthesis agent reliably cites information from the first 15K tokens (web search headlines and snippets) and the final 10K tokens (document analysis conclusions), but frequently omits critical findings that appear in the middle 50K tokens—even when those findings directly address the research question. How should you restructure the aggregated input?

Multi-Agent Research System

Multi-Agent Research System
You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

A
Implement automatic retry logic with exponential backoff within the subagent, returning a generic "search unavailable" status only after all retries are exhausted.

B
Catch the timeout within the subagent and return an empty result set marked as successful.

C
Return structured error context to the coordinator including the failure type, the attempted query, any partial results, and potential alternative approaches.

D
Propagate the timeout exception directly to a top-level handler that terminates the entire research workflow.

A
Stream subagent results to the synthesis agent incrementally, processing web search results first to completion before introducing document analysis findings.

B
Place a key findings summary at the beginning of the aggregated input and organize detailed results with explicit section headers for easier navigation.

C
Summarize all subagent outputs to under 20K tokens total before aggregation, ensuring content stays within the model's reliable processing range.

D
Implement rotation that alternates which subagent's results appear first across different research tasks,

C
Return structured error context to the coordinator including the failure type, the attempted query, any partial results, and potential alternative approaches.

B
Place a key findings summary at the beginning of the aggregated input and organize detailed results with explicit section headers for easier navigation.

Correct!
Returning structured error context—including the failure type, attempted query, partial results, and alternative approaches—gives the coordinator all the information it needs to make intelligent recovery decisions, such as retrying with a modified query or proceeding with partial results. This is the best approach because it preserves maximum context for informed decision-making at the coordination level.

Correct!
Placing a key findings summary at the beginning leverages the primacy effect, ensuring critical information occupies the most reliably attended position. Adding explicit section headers throughout the aggregated input helps the model navigate and attend to middle-section content, directly mitigating the 'lost in the middle' phenomenon.

The document analysis agent discovers that two credible sources contain directly conflicting statistics on a key metric: one government report states 40% growth while an industry analysis states 12% growth. Both sources appear legitimate and the discrepancy could significantly affect the research conclusions. What's the most effective way

24 for the document analysis agent to handle this?

Multi-Agent Research System

Multi-Agent Research System
You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

During a materials research task, the web search subagent queries three source categories with different outcomes: academic databases returned 15 relevant papers, industry reports returned "0 results found," and patent databases returned "Connection timeout." When designing error propagation to the coordinator, what approach enables the

25 best recovery decisions?

Multi-Agent Research System

Multi-Agent Research System
You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

A
Complete the document analysis with both figures included, explicitly annotate the conflict with source attribution, and let the coordinator decide how to reconcile before passing to synthesis.

B
Halt analysis and escalate to the coordinator immediately, asking it to determine which source is authoritative before the agent continues processing remaining documents.

C
Include both figures in the analysis output without flagging them as conflicting, allowing the synthesis agent to determine which to use based on the broader research context.

D
Apply source credibility

A
Complete the document analysis with both figures included, explicitly annotate the conflict with source attribution, and let the coordinator decide how to reconcile before passing to synthesis.

This is the most effective approach because it respects separation of concerns: the document analysis agent completes its primary task without blocking, preserves both conflicting data points with explicit source attribution, and appropriately defers the reconciliation decision to the coordinator, which has the broader context needed to resolve the conflict.

D
Apply source credibility heuristics to select the most likely accurate figure, complete the analysis using that value, and include a footnote mentioning the discrepancy.

Having the document analysis agent apply credibility heuristics to select one figure oversteps its role, as it lacks the broader context needed to make authoritative credibility judgments. This approach also risks losing important information by relegating the discrepancy to a footnote rather than treating it as a significant finding requiring reconciliation.

A
Distinguish access failures (timeout) needing retry decisions from valid empty results ("0 results") representing successful queries.

B
Aggregate outcomes into a single success rate metric (e.g., "67% source coverage") with detailed logs available on request.

C
Report both the timeout and "0 results" as failures requiring coordinator intervention.

D
Have the subagent retry transient failures internally and only report persistent errors.

A
Distinguish access failures (timeout) needing retry decisions from valid empty results ("0 results") representing successful queries.

Correct!
This is correct because a timeout (access failure) and "0 results" (valid empty result) are semantically distinct outcomes requiring different responses. Distinguishing them enables the coordinator to retry the timed-out patent database while accepting the empty industry report results as a valid and informative finding.

A colleague suggests having the document analysis agent send its output directly to the synthesis agent instead of routing through the coordinator. What is the main advantage of keeping the coordinator as the central hub for 26 all subagent communication?

Multi-Agent Research System

Multi-Agent Research System
You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

After running the system on the topic "impact of AI on creative industries," you observe that each subagent completes successfully: the web search agent finds relevant articles, the document analysis agent summarizes papers correctly, and the synthesis agent produces coherent output. However, the final reports cover only visual arts, completely missing music, writing, and film production. When you examine the coordinator's logs, you see it decomposed the topic into three subtasks: "AI in digital art creation," "AI in graphic design," and "AI in 27 photography." What is the most likely root cause?

Multi-Agent Research System

Multi-Agent Research System
You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

A
Subagents operate with isolated memory, and direct communication would require complex serialization that only the coordinator can perform

B
Routing through the coordinator enables automatic retry logic that direct agent-to-agent calls cannot support

C
The coordinator can observe all interactions, handle errors consistently, and decide what information each subagent should receive

D
The coordinator batches multiple subagent requests together, reducing the total number of API calls and overall latency

C
The coordinator can observe all interactions, handle errors consistently, and decide what information each subagent should receive

Correct!
This is correct. The coordinator pattern provides centralized visibility into all interactions, consistent error handling across the system, and fine-grained control over what information each subagent receives, which are the primary advantages of hub-and-spoke communication.

A
The coordinator agent's task decomposition is too narrow, resulting in subagent assignments that don't cover all relevant domains of the topic.

B
The web search agent's queries are not comprehensive enough and need to be expanded to cover more creative industry sectors.

C
The document analysis agent is filtering out sources related to non-visual creative industries due to overly restrictive relevance criteria.

D
The synthesis agent lacks instructions for identifying coverage gaps in the findings it receives from other agents.

The coordinator's logs directly reveal it decomposed the broad topic into only three visual arts subtasks (digital art, graphic design, photography), completely omitting music, writing, and film. Since the subagents all executed their assigned tasks correctly, the narrow decomposition by the coordinator is clearly the root cause of the missing coverage.

A
The coordinator agent's task decomposition is too narrow, resulting in subagent assignments that don't cover all relevant domains of the topic.

During testing, you observe that the synthesis agent frequently needs to verify specific claims while combining findings. Currently, when verification is needed, the synthesis agent returns control to the coordinator, which invokes the web search agent, then re-invokes synthesis with results. This adds 2-3 round trips per task and increases latency by 40%. Your evaluation shows that 85% of these verifications are simple fact-checks (dates, names, statistics) while 15% require deeper investigation. What's the most effective approach to reduce overhead while maintaining system reliability?

Multi-Agent Research System

Multi-Agent Research System
You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

Production logs reveal a consistent pattern: requests to "analyze the quarterly report I uploaded" are routed to the web search agent 45% of the time instead of the document analysis agent. Examining the tool definitions, you find the web search agent has an analyze_content tool described as "analyzes content and extracts key information," while the document analysis agent has an analyze_document tool described as "analyzes documents and extracts key information." How should you address this misrouting?

Multi-Agent Research System

Multi-Agent Research System
You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

A
Have the web search agent proactively cache extra context around each source during initial research, anticipating what the synthesis agent might need to verify.

B
Give the synthesis agent access to all web search tools so it can handle any verification need directly without round-trips through the coordinator.

C
Have the synthesis agent accumulate all verification needs and return them as a batch to the coordinator at the end of its pass, which then sends them all to the web search agent at once.

D
Give the synthesis agent a scoped verify_fact tool for simple lookups, while complex

A
Expand the document analysis tool's description to include example use cases like "Use for uploaded PDFs, Word documents, and spreadsheets" while leaving the web search tool unchanged.

B
Add a pre-routing classifier that determines whether the user is referencing uploaded files or web content before the coordinator makes delegation decisions.

C
Rename the web search tool to extract_web_results and update its description to "processes and returns information retrieved from web searches and URLs."

D
Add few-shot examples to the coordinator's prompt showing correct routing: "User uploads

D
Give the synthesis agent a scoped verify_fact tool for simple lookups, while complex verifications continue delegating to the web search agent through the coordinator.

C
Rename the web search tool to extract_web_results and update its description to "processes and returns information retrieved from web searches and URLs."

Correct!
Providing a scoped fact-verification tool handles the 85% of simple lookups directly, eliminating most round-trips while preserving the coordinator-based delegation path for the 15% of complex verifications. This applies the principle of least privilege, keeping the synthesis agent focused on its primary task while still reducing latency significantly.

Correct!
Renaming the web search tool to `extract\_web\_results` and updating its description to clearly reference web searches and URLs directly addresses the root cause by eliminating the semantic overlap between the two tools' names and descriptions. This makes each tool's purpose unambiguous, allowing the coordinator to correctly distinguish between document analysis and web search tasks.

When designing the system, you gave the document analysis agent access to a general-purpose `fetch_url` tool so it could load documents from URLs. Production logs reveal this agent now frequently fetches search engine result pages to conduct ad-hoc web searches—behavior that should route through the web search agent. This

Multi-Agent Research System

Multi-Agent Research System

You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

You're creating a custom /explore-alternatives skill that your team uses to brainstorm and evaluate different implementation approaches before committing to one. However, developers report that after running this skill, Claude's subsequent responses are influenced by the exploration discussion—sometimes referencing abandoned approaches or maintaining exploratory context that confuses actual implementation work. What's the

Code Generation with Claude Code

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, `CLAUDE.md` configurations, and understand when to use plan mode vs direct execution.

Your CLAUDE.md has grown to over 400 lines containing coding standards, testing conventions, a detailed PR review checklist, deployment workflow instructions, and database migration procedures. You want Claude to always follow the coding standards and testing conventions, but only apply PR review, deployment, and migration guidance when you're actually performing those 32 tasks. What's the most effective restructuring approach?

Code Generation with Claude Code

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, `CLAUDE.md` configurations, and understand when to use plan mode vs direct execution.

A

Add instructions to the document analysis agent's prompt clarifying it should only use `fetch_url` for loading document URLs, not searching.

B

Remove `fetch_url` from the document analysis agent and route all URL loading through the coordinator to the web search agent.

C

Implement filtering that blocks `fetch_url` calls to known search engine domains while allowing other URLs.

D

Replace `fetch_url` with a `load_document` tool that validates URLs point to document formats.

□

Replace `fetch_url` with a `load_document` tool that validates URLs point to document formats.

Correct!
Replacing the general-purpose tool with a document-specific tool that validates URLs point to document formats addresses the root cause by constraining capability at the interface level. This follows the principle of least privilege, making the undesired search behavior impossible rather than merely discouraged.

A

Use the **!** prefix in the skill to execute the exploration logic as a bash subprocess.

B

Add context: fork to the skill's frontmatter.

C

Create the skill in
~/.claude/skills/ instead of
.claude/skills/.

D

Split the skill into two separate skills—/explore-start and /explore-end—to demarcate when exploration context should be discarded.

Add context: fork to the skill's frontmatter.

Correct!
The 'context: fork' frontmatter option runs the skill in an isolated sub-agent context, so the exploration discussion does not pollute the main conversation history. This prevents abandoned approaches and exploratory context from influencing subsequent implementation work.

A

Keep universal standards in CLAUDE.md and create Skills for task-specific workflows (PR reviews, deployments, migrations) with trigger keywords

B

Split the CLAUDE.md into files in `.claude/rules/` with path-specific glob patterns so each rule loads only for matching file types

C

Keep all content in CLAUDE.md
but use @import syntax to
organize it into separately
maintained files by category

D

Move all guidance into separate Skills files organized by workflow type, keeping only a brief project description in CLAUDE.md

A

- Keep universal standards in Claude.md and create Skills for task-specific workflows (PR reviews, deployments, migrations) with trigger keywords

Correct! This is the most effective approach because CLAUDE.md content is loaded for every conversation, ensuring coding standards and testing conventions are always applied, while Skills are invoked on-demand when Claude detects relevant trigger keywords, making them ideal for task-specific workflows like PR reviews, deployments, and migrations.

You've been assigned to restructure the team's monolithic application into microservices. This will involve changes across dozens of files and requires decisions about service boundaries and module dependencies. Which approach should you take?

Code Generation with Claude Code

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

Your team created an /analyze-codebase skill that performs comprehensive code analysis—dependency scanning, test coverage calculation, and code quality metrics. After running this command, team members report that Claude becomes less responsive in the session and loses track of their original task. What's the most effective way to address this while preserving full analysis capability?

Code Generation with Claude Code

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

Your codebase has distinct areas with different coding conventions: React components use functional style with hooks, API handlers use async/await with specific error handling, and database models follow a repository pattern. Test files are spread throughout the codebase alongside the code they test (e.g., Button.test.tsx next to Button.tsx), and you want all tests to follow the same conventions regardless of location. What's the most maintainable way to ensure Claude automatically applies the correct conventions when generating code?

Code Generation with Claude Code

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

A
Start with direct execution and make changes incrementally, letting the implementation reveal the natural service boundaries.

B
Use direct execution with comprehensive upfront instructions detailing exactly how each service should be structured.

C
Begin in direct execution mode and only switch to plan mode if you encounter unexpected complexity during implementation.

D
Enter plan mode to explore the codebase, understand dependencies, and design an implementation approach before making changes.

A
Split the skill into three smaller skills that each generate less output

B
Add context: fork to the skill's frontmatter to run the analysis in an isolated sub-agent context

C
Add model: haiku to the frontmatter to use a faster, more efficient model for the analysis

D
Add instructions to the skill to compress all outputs into a brief summary before displaying

A
Create skills in .claude/skills/ for each code type that include the relevant conventions in their SKILL.md files

B
Create rule files in .claude/rules/ with YAML frontmatter specifying glob patterns to conditionally apply conventions based on file paths

C
Place a separate CLAUDE.md file in each subdirectory containing that area's specific conventions

D
Consolidate all conventions in the root CLAUDE.md file under headers for each area, relying on Claude to infer which section applies

Correct!
Using plan mode to explore the codebase, understand dependencies, and design an approach before making changes is the correct strategy for a complex architectural restructuring like breaking apart a monolith. This allows safe exploration and informed decision-making about service boundaries before committing to potentially costly changes across dozens of files.

D
Enter plan mode to explore the codebase, understand dependencies, and design an implementation approach before making changes.

B
Add context: fork to the skill's frontmatter to run the analysis in an isolated sub-agent context
This preserves full analysis capability while keeping the main session responsive.

Correct!
Using rule files in '.claude/rules/' with YAML frontmatter and glob patterns (e.g., '**/*.test.tsx', 'src/api/**/*.ts') allows conventions to be automatically and deterministically applied based on file paths, regardless of where those files are located in the directory structure. This is the most maintainable approach because it handles cross-cutting concerns like test files spread throughout the codebase without requiring duplication or manual intervention.

Your team has created a /migration skill that generates database migration files. The skill accepts a migration name via \$ARGUMENTS. In production, you're seeing three issues: (1) developers often invoke the skill without arguments, resulting in poorly-named files, (2) the skill sometimes incorporates database schema details from unrelated earlier conversations, and (3) a developer accidentally triggered destructive test cleanup when the skill had broad tool access. Which configuration approach addresses all three issues?

Code Generation with Claude Code

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

Your team has been using Claude Code for several months. Recently, three developers report that Claude correctly follows your "always include comprehensive error handling" guideline, but a fourth developer who just joined reports Claude isn't following this guideline. All four developers are working in the same repository and have the latest code pulled. What's the most likely cause and appropriate fix?

Code Generation with Claude Code

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

A
Split into separate /migration-create and /migration-apply skills, add instructions in each SKILL.md to request a migration name if not provided, and use different allowed-tools scopes for each skill.

B
Use positional parameters \$1 and \$2 instead of \$ARGUMENTS to enforce specific inputs, include explicit schema file references via @ syntax to control context, and add description frontmatter warning about destructive operations.

C
Include validation instructions in the skill's SKILL.md that direct Claude to verify \$ARGUMENTS contains a valid name, add prompts to ignore prior conversation context, and list forbidden operations Claude should avoid.

This approach correctly uses three distinct skill configuration features to address each issue: 'argument-hint' frontmatter shows expected parameters during autocomplete (addressing missing arguments), 'context: fork' isolates execution in a subagent context separate from conversation history (preventing context bleeding from earlier conversations), and 'allowed-tools' restricts tool access to only file write operations (preventing destructive actions).

C
Include validation instructions in the skill's SKILL.md that direct Claude to verify \$ARGUMENTS contains a valid name, add prompts to ignore prior conversation context, and list forbidden operations Claude should avoid.

This approach relies entirely on prompt-based instructions, which are unreliable for enforcement—telling Claude to "ignore prior context" doesn't actually isolate execution context, and listing forbidden operations doesn't restrict tool access the way 'allowed-tools' does, leaving the system vulnerable to all three issues.

A
The guideline exists in the original developers' ~/.claude/CLAUDE.md files (user-level) instead of the project's .claude/CLAUDE.md. Move the instruction to the project-level file so all team members receive it.

B
Claude Code builds per-user preference models over time through repeated interactions. The new developer needs to repeatedly specify the error handling requirement until Claude learns their preferences.

Correct!
This is the most likely cause: if the error handling guideline was added to each original developer's user-level ~/.claude/CLAUDE.md rather than the project's .claude/CLAUDE.md, new team members would not receive it. Moving the instruction to the project-level configuration file ensures all current and future team members automatically receive the guideline.

C
The new developer's ~/.claude/CLAUDE.md contains conflicting instructions that override the project settings. Have them remove the conflicting section from their user-level

A
The guideline exists in the original developers' ~/.claude/CLAUDE.md files (user-level) instead of the project's .claude/CLAUDE.md. Move the instruction to the project-level file so all team members receive it.

You've asked Claude Code to implement a function that transforms API responses into a normalized internal format. After two iterations, the output structure still doesn't match expectations—some fields are nested differently and timestamps aren't formatted correctly. You've been describing the requirements in prose, but Claude seems to interpret them differently each time.

38 What's the most effective approach for the next iteration?

Code Generation with Claude Code

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

You've created a /commit skill in .claude/skills/commit/SKILL.md that your team uses. One developer wants to customize it for their personal workflow (different commit message format, additional checks) without affecting teammates. What should you recommend?

Code Generation with Claude Code

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

A

Write a JSON schema defining the expected output structure and validate Claude's output against it after each iteration.

B

Rewrite your requirements with greater technical precision, specifying exact field mappings, nesting rules, and timestamp format strings.

C

Ask Claude to explain its current interpretation of the requirements so you can identify where understanding diverges.

D

Provide 2-3 concrete input-output examples showing the expected transformation for representative API responses.

D

Provide 2-3 concrete input-output examples showing the expected transformation for representative API responses.

Correct!

Providing concrete input-output examples is the most effective approach because it eliminates the ambiguity inherent in prose descriptions by showing Claude exactly what the expected transformation looks like. This directly addresses the root cause—misinterpretation of prose requirements—by giving unambiguous, concrete targets for field nesting and timestamp formatting.

A

Create a personal version in ~/.claude/skills/ with a different name like /my-commit

B

Set override: true in the personal skill's frontmatter to take precedence over the project version

C

Create a personal version at ~/.claude/skills/commit/SKILL.md with the same name

D

Add username-based conditional logic to the project skill's frontmatter

A

Create a personal version in ~/.claude/skills/ with a different name like /my-commit

Correct!

This is correct. Since project skills take precedence over personal skills with the same name, the developer must use a different skill name (like ' /my-commit ') in their personal '~/.claude/skills/' directory to ensure their custom version is accessible alongside the team's project skill.

A	Use the Explore subagent for Phase 1 to isolate verbose output and return a summary, then continue Phases 2-3 in the main conversation.	
	B Switch to headless mode with --continue, passing explicit context summaries between batch invocations to maintain continuity.	
	C Continue all phases in the main conversation, using /compact periodically to reduce context usage as you progress through the files.	
	D Define your error handling pattern in CLAUDE.md, then process files in batches across multiple sessions, relying on the shared memory file for consistency.	
A	Use the Explore subagent for Phase 1 to isolate verbose output and return a summary, then continue Phases 2-3 in the main conversation.	Correct! Using the Explore subagent for Phase 1 is ideal because it isolates the verbose discovery output in a separate context, returning only a concise summary to the main conversation. This preserves the main context window for the collaborative design and consistent implementation phases where retained context is most valuable.
A	In ~/.claude/commands/ in each developer's home directory	
	B In the CLAUDE.md file at the project root	
	C In the .claude/commands/ directory in the project repository	
	D In a .claude/config.json file with a commands array	
A	In the .claude/commands/ directory in the project repository	Correct! Placing custom slash commands in the ~/.claude/commands/ directory within the project repository is correct because these files are version-controlled and automatically available to every developer who clones or pulls the repo. This is the designated location for project-scoped custom commands in Claude Code.
A	Enter plan mode to explore the integration options and their architectural implications, then present a recommendation before implementing.	
	B Start direct execution to scaffold the Slack channel class following existing patterns, deferring the integration method decision until later.	
	C Start direct execution using the bot token approach to enable delivery confirmation capabilities.	
	D Start direct execution using incoming webhooks to match the existing one-way notification pattern.	
A	Enter plan mode to explore the integration options and their architectural implications, then present a recommendation before implementing.	Correct! This is correct because the Slack integration involves multiple valid approaches with significantly different architectural implications, and the requirements are ambiguous. Using plan mode to explore trade-offs between webhooks, bot tokens, and Slack Apps allows for an informed recommendation and team alignment before committing to an implementation path

You're adding error handling wrappers to external API calls across a 120-file codebase. The task has three phases: (1) discovering all API call locations and patterns, (2) designing the error handling approach collaboratively, and (3) implementing wrappers consistently. During Phase 1, Claude generates verbose output listing hundreds of call sites with context. Your context window is filling rapidly before you've finished discovery.

What's the most effective approach to complete this while maintaining implementation consistency?

Code Generation with Claude Code

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

You want to create a custom /review slash command that runs your team's standard code review checklist. This command should be available to every developer when they clone or pull the repository. Where should you create this command file?

Code Generation with Claude Code

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

You need to add Slack as a new notification channel. The existing codebase has clear, consistent patterns for email, SMS, and push channels. However, the Slack API offers fundamentally different integration approaches—incoming webhooks (simple, one-way only), bot tokens (enables delivery confirmation and programmatic control), or Slack Apps (bidirectional events, requires workspace approval). Your ticket says "add Slack support" without specifying which integration method or whether advanced features like delivery tracking will be needed. How should you approach this task?

Code Generation with Claude Code

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

Your team's CLAUDE.md file has grown to over 500 lines, mixing TypeScript conventions, testing guidelines, API patterns, and deployment procedures. Developers find it difficult to locate and update relevant sections. What approach does Claude Code support for organizing project-level instructions into focused, topic-specific modules?

Code Generation with Claude Code

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

Your team wants to add a GitHub MCP server to enable PR lookups and CI status checks through Claude Code. Each of the six developers has their own GitHub personal access token. You want consistent tooling across the team without committing credentials to version control. What's the most effective configuration approach?

Code Generation with Claude Code

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

A

Create separate markdown files in `.claude/rules/`, each covering one topic (e.g., `testing.md`, `api-conventions.md`)

B

Create multiple files named `CLAUDE.md` at different levels of the directory tree, each one overriding the parent's instructions

C

Split instructions into `README.md` files in relevant subdirectories, which Claude automatically loads as instructions

D

Define a `.claude/config.yaml` file that maps file patterns to specific sections within `CLAUDE.md`

A

Create an MCP server wrapper that reads tokens from a `.env` file and proxies requests to the GitHub API, then add this wrapper to your project `.mcp.json`.

B

Configure the server in project scope with a placeholder token value, then instruct developers to override it in their local scope configuration.

C

Add the server to a project-scoped `.mcp.json` with environment variable expansion `(${GITHUB_TOKEN})` for authentication, and document the required environment variable in your project `README`.

D

Have each developer configure the server in user scope with `claude mcp add --scope user`.

A

Create separate markdown files in `.claude/rules/`, each covering one topic (e.g., `testing.md`, `api-conventions.md`)

C

Add the server to a project-scoped `.mcp.json` with environment variable expansion `(${GITHUB_TOKEN})` for authentication, and document the required environment variable in your project `README`.

Correct!
This is correct. Claude Code supports a `.claude/rules/` directory where you can create separate markdown files for topic-specific guidelines (e.g., `testing.md`, `api-conventions.md`), allowing teams to organize large instruction sets into focused, maintainable modules.

Correct!
Using a project-scoped `.mcp.json` with environment variable expansion `(' ${GITHUB_TOKEN} ')` is the idiomatic approach—it provides a single, version-controlled source of truth for the team's MCP configuration while allowing each developer to supply their own credentials through environment variables. Documenting the required variable in the `README` ensures easy onboarding without ever committing secrets.

You've found that including 2-3 full exemplar endpoint implementations as context significantly improves consistency when generating new API endpoints. However, this context is only useful for creating new endpoints—not for bug fixes, code reviews, or other API directory work.

45 What's the most efficient configuration approach?

Code Generation with Claude Code

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

Production logs show that for simple requests like "refund order #1234", your agent succeeds in 3-4 tool calls with 91% resolution rate. However, for complex requests like "I've been charged twice, my discount didn't apply, and I want to cancel", the agent averages 12+ tool calls with only 54% resolution—often investigating concerns sequentially and gathering redundant customer data for each one.

What's the most effective change to improve complex request handling?

Customer Support Resolution Agent

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (get_customer, lookup_order, process_refund, escalate_to_human). Your target is 80%+ first-contact resolution while knowing when to escalate.

A
Add the exemplar endpoint code with pattern documentation to the project CLAUDE.md file so it's automatically available.

B
Create a skill that references the exemplar endpoints and includes pattern-following instructions, invoked on-demand via slash command.

C
Reference the exemplar endpoints manually in each generation request by copying relevant code into your prompt.

D
Configure path-specific rules in .claude/rules/api/ that include the exemplar code and activate when working in the API directory.

B
Create a skill that references the exemplar endpoints and includes pattern-following instructions, invoked on-demand via slash command.

Creating a skill with the exemplar endpoints and pattern-following instructions allows on-demand invocation via a slash command, ensuring the context is loaded only when generating new endpoints and not during unrelated tasks like bug fixes or code reviews.

D
Configure path-specific rules in .claude/rules/api/ that include the exemplar code and activate when working in the API directory.

Path-specific rules that activate when working in the API directory would trigger for all API directory work, including bug fixes and code reviews—exactly the scenarios where the exemplar context is unnecessary and wasteful.

A
Add explicit verification gates between steps requiring the agent to checkpoint after resolving each concern before moving to the next.

B
Add few-shot examples demonstrating ideal tool call sequences for various multi-part billing scenarios to your system prompt.

C
Reduce the number of available tools by consolidating get_customer, lookup_order, and billing-related lookups into a single investigate_issue tool.

D
Decompose the request into distinct concerns, then investigate each in parallel using shared customer context before synthesizing a resolution.

D
Decompose the request into distinct concerns, then investigate each in parallel using shared customer context before synthesizing a resolution.

Decomposing the request into distinct concerns and investigating them in parallel with shared customer context directly addresses both core issues: it eliminates redundant data fetching by reusing context across concerns and reduces total tool calls by parallelizing investigations before synthesizing a unified resolution.

B
Add few-shot examples demonstrating ideal tool call sequences for various multi-part billing scenarios to your system prompt.

While few-shot examples might marginally improve tool call sequences for known scenarios, they don't systematically solve the structural problems of redundant context gathering or sequential investigation, and they won't generalize well to novel multi-part request combinations.

Production logs reveal a consistent pattern: when customers include "account" in messages (e.g., "I want to check my account for the order I placed yesterday"), the agent calls `get_customer` first 78% of the time. When customers phrase similar requests without "account" (e.g., "I want to check on the order I placed yesterday"), it calls `lookup_order` first 93% of the time. The tool descriptions are well-written and unambiguous. What is the most likely root cause of this discrepancy?

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (`get_customer`, `lookup_order`, `process_refund`, `escalate_to_human`). Your target is 80%+ first-contact resolution while knowing when to escalate.

Customer Support Resolution Agent

In testing, you notice the agent frequently calls `get_customer` when users ask about order status, even though `lookup_order` would be more appropriate. What should you examine first to address this issue?

Customer Support Resolution Agent

A

The system prompt contains keyword-sensitive instructions that steer behavior based on terms like "account," creating unintended tool selection patterns

B

The tool descriptions need additional negative examples specifying when NOT to use each tool to prevent this keyword-triggered confusion

C

The model requires more training data on multi-concept messages and should be fine-tuned on examples that include both account and order language

D

The model's base training creates associations between "account" terminology and customer-related operations

A

The system prompt contains keyword-sensitive instructions that steer behavior based on terms like "account," creating unintended tool selection patterns

This is the most likely root cause because the systematic, keyword-triggered pattern (78% vs 93%) strongly suggests explicit routing logic in the system prompt that reacts to the word "account" and directs the agent toward customer-related tools. Since the tool descriptions are already well-written and unambiguous, the discrepancy points to prompt-level instructions creating unintended behavioral steering.

A

Add few-shot examples covering every possible order-related query pattern to the system prompt

B

Reduce the number of tools available to the agent to simplify selection

C

Implement a pre-processing classifier that detects order queries and routes directly to `lookup_order`

D

Review tool descriptions to ensure they clearly distinguish each tool's purpose

D

Review tool descriptions to ensure they clearly distinguish each tool's purpose

Tool descriptions are the primary input the model uses to decide which tool to call. When an agent consistently selects the wrong tool, the first diagnostic step is to examine whether the tool descriptions clearly distinguish each tool's purpose and specify when each should be used.

Your agent achieves 55% first-contact resolution, well below the 80% target. Logs show it escalates straightforward cases (standard damage replacements with photo evidence) while attempting to autonomously handle complex situations requiring policy exceptions. What's the most effective way to improve escalation calibration?

Customer Support Resolution Agent

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (get_customer, lookup_order, process_refund, escalate_to_human). Your target is 80%+ first-contact resolution while knowing when to escalate.

Production logs show the agent sometimes selects get_customer when lookup_order would be more appropriate, particularly for ambiguous requests like "I need help with my recent purchase." You decide to add few shot examples to your system prompt to improve tool selection. Which approach will most effectively address this issue?

Customer Support Resolution Agent

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (get_customer, lookup_order, process_refund, escalate_to_human). Your target is 80%+ first-contact resolution while knowing when to escalate.

A
Implement sentiment analysis to detect customer frustration levels and automatically escalate when negative sentiment exceeds a threshold.

B
Deploy a separate classifier model trained on historical tickets to predict which requests need escalation before the main agent begins processing.

C
Add explicit escalation criteria to your system prompt with few-shot examples demonstrating when to escalate versus resolve autonomously.

D
Have the agent self-report a confidence score (1-10) before each response and automatically route requests to humans when confidence falls below a threshold.

C
Add explicit escalation criteria to your system prompt with few-shot examples demonstrating when to escalate versus resolve autonomously.

Adding explicit escalation criteria with few-shot examples directly addresses the root cause—unclear decision boundaries between straightforward and complex cases. This is the most proportionate and effective first intervention, as it teaches the agent precisely when to escalate versus resolve autonomously without requiring additional infrastructure.

A
Add 4-6 examples targeting ambiguous scenarios, each showing reasoning for why one tool was chosen over plausible alternatives.

B
Add examples grouped by tool—all get_customer scenarios together, then all lookup_order scenarios.

C
Add 10-15 examples of clear, unambiguous requests that demonstrate correct tool selection for each tool's typical use cases.

D
Add explicit "use when" and "do not use when" guidelines in each tool's description covering the ambiguous cases.

A
Add 4-6 examples targeting ambiguous scenarios, each showing reasoning for why one tool was chosen over plausible alternatives.

Targeting few-shot examples at the specific ambiguous scenarios where errors occur, with explicit reasoning about why one tool is preferred over another, directly teaches the model the comparative decision-making process it needs for edge cases. This approach is the most effective because worked examples demonstrating reasoning are better than declarative rules for nuanced tool selection.

Your agent handles single-concern requests with 94% accuracy (e.g., "I need a refund for order #1234"). However, when customers include multiple concerns in one message (e.g., "I need a refund for order #1234 and also want to update my shipping address for order #5678"), tool selection accuracy drops to 58%. The agent typically addresses only one concern or mixes up parameters between requests. What's the most effective approach to improve reliability for multi-concern requests?

51

Customer Support Resolution Agent

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (get_customer, lookup_order, process_refund, escalate_to_human). Your target is 80%+ first-contact resolution while knowing when to escalate.

Production data shows that in 12% of cases, your agent skips get_customer entirely and calls lookup_order using only the customer's stated name, occasionally leading to misidentified accounts and incorrect refunds. What change would most effectively address this reliability issue?

52

Customer Support Resolution Agent

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (get_customer, lookup_order, process_refund, escalate_to_human). Your target is 80%+ first-contact resolution while knowing when to escalate.

A

Implement a preprocessing layer that uses a separate model call to decompose multi-concern messages into individual requests, process each independently, then combine the results.

B

Add few-shot examples to your prompt demonstrating the correct reasoning and tool sequence for multi-concern requests.

C

Consolidate related tools into fewer, more general-purpose tools.

D

Implement response validation that detects incomplete responses and automatically re-prompts the agent to address any missed concerns.

B

Add few-shot examples to your prompt demonstrating the correct reasoning and tool sequence for multi-concern requests.

Adding few-shot examples demonstrating correct reasoning and tool sequencing for multi-concern requests is the most effective approach because the agent already handles individual concerns well at 94% accuracy—it simply needs pattern guidance for handling multiple concerns in one message. This is a low-cost, proven technique that directly addresses the root cause of the agent failing to decompose and properly route parameters across multiple requests.

A

Implement a preprocessing layer that uses a separate model call to decompose multi-concern messages into individual requests, process each independently, then combine the results.

Using a separate model call to decompose multi-concern messages adds unnecessary latency, complexity, and cost when the agent already demonstrates strong single-concern understanding. This over-engineers the solution when simpler prompt-level guidance can address the pattern recognition gap.

A

Implement a routing classifier that analyzes each request and enables only the subset of tools appropriate for that request type.

B

Add a programmatic prerequisite that blocks lookup_order and process_refund calls until get_customer has returned a verified customer ID.

C

Add few-shot examples showing the agent always calling get_customer first, even when customers volunteer order details.

D

Enhance the system prompt to state that customer verification via get_customer is mandatory before any order operations.

B

Add a programmatic prerequisite that blocks lookup_order and process_refund calls until get_customer has returned a verified customer ID.

Adding a programmatic prerequisite that blocks downstream tools until 'get_customer' returns a verified customer ID provides a deterministic guarantee that the required sequence is followed. This is the most effective approach because it removes the possibility of the agent skipping verification, regardless of LLM behavior.

Production logs reveal that the agent misinterprets data from your MCP tools: Unix timestamps from `get_customer`, ISO 8601 dates from `lookup_order`, and numeric status codes (1=pending, 2=shipped). Some tools are third-party MCP servers you cannot modify. What's the most maintainable approach to normalize data formats?

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (`get_customer`, `lookup_order`, `process_refund`, `escalate_to_human`). Your target is 80%+ first-contact resolution while knowing when to escalate.

Customer Support Resolution Agent

You're implementing the agentic loop for your support agent. After each API call to Claude, you need to determine whether to continue the loop (execute the requested tools and call Claude again) or stop (present the final response to the customer). What determines this decision?

Customer Support Resolution Agent

A
Modify tools you control to return human-readable formats; create wrapper tools for third-party tools

B
Create a `normalize_data` tool that the agent calls after each data retrieval to transform values

C
Add detailed format documentation to your system prompt explaining each tool's data conventions

D
Use a `PostToolUse` hook to intercept tool results and apply formatting transformations before agent processing

D
Use a `PostToolUse` hook to intercept tool results and apply formatting transformations before agent processing

Using a `PostToolUse` hook provides a centralized, deterministic point to intercept and normalize all tool outputs—including those from third-party MCP servers—before the agent processes them. This is the most maintainable approach because it applies transformations uniformly via code rather than relying on LLM interpretation or agent behavior.

A
Parse Claude's response text for phrases like "I've completed" or "Is there anything else?"—these natural language signals indicate the task is finished.

B
Set a maximum iteration count (e.g., 10 calls) and stop when reached, regardless of whether Claude indicates more work is needed.

C
Check the `stop_reason` field in Claude's response—continue when it equals "tool_use" and stop when it equals "end_turn".

D
Check whether the response includes any assistant text content—if Claude generated explanatory text, the loop should end.

This is correct. The `'stop_reason'` field is Claude's explicit, structured signal for loop control: `'tool_use'` indicates Claude wants to execute a tool and receive the results back, while `'end_turn'` indicates Claude has completed its response and the loop should terminate.

Your support agent uses progressive summarization—when context reaches 70% capacity, older turns are summarized while recent ones remain verbatim. Production logs reveal a pattern: customers reference specific amounts ("the 15% discount I mentioned"), but the agent responds with incorrect values. Investigation shows these details were stated 20+ turns ago and got condensed into vague summaries like "discussed promotional pricing." What's the most effective

55 fix?

Customer Support Resolution Agent

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (get_customer, lookup_order, process_refund, escalate_to_human). Your target is 80%+ first-contact resolution while knowing when to escalate.

Your get_customer tool returns all matches when searching by name. Claude currently picks the customer with the most recent order when multiple results are returned, but production data shows this causes 15% of multi-match cases to proceed with the wrong customer

56 account. How should you address this?

Customer Support Resolution Agent

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (get_customer, lookup_order, process_refund, escalate_to_human). Your target is 80%+ first-contact resolution while knowing when to escalate.

A

Revise the summarization prompt to explicitly preserve all numerical values, percentages, dates, and customer-stated expectations verbatim.

B

Store full conversation history externally and implement retrieval to search it when the agent detects reference phrases like "as I mentioned."

C

Extract transactional facts (amounts, dates, order numbers) into a persistent "case facts" block included in each prompt, outside the summarized history.

D

Increase the summarization threshold from 70% to 85% capacity so conversations have more room before summarization triggers.

A

Modify get_customer to return only the single most likely match based on a ranking algorithm, simplifying Claude's decision by eliminating ambiguous results.

B

Implement a confidence scoring system that proceeds automatically above 85% confidence and prompts for clarification below that threshold.

C

Instruct Claude to ask for an additional identifier (email, phone, or order number) when get_customer returns multiple matches, before taking any customer-specific action.

D

Add few-shot examples showing Claude how to use conversational context (products mentioned, dates

Extracting transactional facts (amounts, dates, order numbers) into a persistent "case facts" block addresses the root cause: summarization is inherently lossy for precise details. By preserving critical information in a structured block outside the summarized history, these facts remain reliably available in every prompt regardless of how many turns are summarized.

C

Extract transactional facts (amounts, dates, order numbers) into a persistent "case facts" block included in each prompt, outside the summarized history.

Asking the user for an additional identifier (such as email, phone, or order number) is the most reliable way to disambiguate multiple matches, since the user has definitive knowledge of their own identity. One extra conversational turn is a small cost to eliminate the 15% error rate caused by incorrect customer selection.

C

Instruct Claude to ask for an additional identifier (email, phone, or order number) when get_customer returns multiple matches, before taking any customer-specific action.

Production metrics show that when your agent resolves complex cases involving billing disputes or multi-order returns, customer satisfaction scores are 15% lower than for simple cases—even when the resolution is technically correct. Root cause analysis reveals the agent provides accurate resolutions but inconsistently explains the reasoning: sometimes omitting relevant policy details, other times missing timeline information or next steps. The specific context gaps vary by case. You want to improve resolution quality without adding human review overhead.

57 Which approach is most effective?

Customer Support Resolution Agent

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (get_customer, lookup_order, process_refund, escalate_to_human). Your target is 80%+ first-contact resolution while knowing when to escalate.

Production metrics show your agent averages 4+ API round-trips per resolution. Analysis reveals Claude frequently requests get_customer and lookup_order in separate sequential turns even when both are needed upfront.

58 What's the most effective way to reduce round-trips?

Customer Support Resolution Agent

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (get_customer, lookup_order, process_refund, escalate_to_human). Your target is 80%+ first-contact resolution while knowing when to escalate.

A

Add a self-critique step where the agent evaluates its draft response for completeness—ensuring it addresses the customer's concern, includes relevant context, and anticipates follow-up questions.

B

Add a confirmation step where the agent asks "Does this fully address your concern?" before closing, letting customers request additional information if needed.

C

Implement few-shot examples in the system prompt showing complete resolution explanations for five common complex case types, demonstrating how to include policy context, timelines, and next steps.

D

A

Add a self-critique step where the agent evaluates its draft response for completeness—ensuring it addresses the customer's concern, includes relevant context, and anticipates follow-up questions.

A self-critique step (evaluator-optimizer pattern) directly addresses the root cause of inconsistent explanation completeness by having the agent evaluate its own draft against specific criteria—such as policy context, timelines, and next steps—before presenting it. This catches case-specific gaps that vary across different complex scenarios without requiring human review.

A

Increase max_tokens to give Claude more space to plan ahead and naturally batch its tool requests.

B

Prompt Claude to batch tool requests per turn, and return all tool results together before the next API call.

C

Implement speculative execution that automatically calls likely-needed tools alongside any requested tool, returning all results regardless of what was requested.

D

Create composite tools like get_customer_with_orders that bundle common lookup combinations into single calls.

B

Prompt Claude to batch tool requests per turn, and return all tool results together before the next API call.

Prompting Claude to batch related tool requests in a single turn, and returning all results together before the next API call, leverages Claude's native ability to request multiple tools simultaneously. This is the most effective approach because it directly addresses the sequential calling pattern with minimal architectural changes.

A

The customer claims they never received their order, but tracking shows it was delivered and signed for at their address three days ago. The agent should escalate because presenting contradictory evidence might damage the customer relationship.

B

The customer requests a price match against a competitor. Your policies allow adjustments for price drops on your own site within 14 days but are silent on competitor pricing. The agent should escalate for policy interpretation.

B

The customer requests a price match against a competitor. Your policies allow adjustments for price drops on your own site within 14 days but are silent on competitor pricing. The agent should escalate for policy interpretation.

This represents a genuine policy gap where the company's guidelines cover own-site price drops but are silent on competitor price matching, meaning the agent cannot fabricate a policy and must escalate for human judgment on how to interpret or extend existing rules.

C

The customer wants to cancel an order that shipped yesterday, with delivery scheduled for tomorrow. The agent should escalate because

A

Consolidate both tools into a single `lookup_entity` tool that accepts any identifier and internally determines which backend to query.

B

Expand each tool's description to include input formats it handles, example queries, edge cases, and boundaries explaining when to use it versus similar tools.

Expanding tool descriptions to include input formats, example queries, edge cases, and boundaries directly addresses the root cause—minimal descriptions that leave the LLM unable to distinguish between similar tools. This is a low-effort, high-leverage first step that improves the primary mechanism LLMs use for tool selection.

C

Implement a routing layer that parses user input before each turn and pre-selects the appropriate tool based on detected keywords and identifier patterns.

B

Expand each tool's description to include input formats it handles, example queries, edge cases, and boundaries explaining when to use it versus similar tools.

D

Add few-shot examples to the system prompt demonstrating correct tool selection patterns, with 5-8 examples showing

After calling `get_customer` and `lookup_order`, the agent has retrieved all available system data but faces uncertainty. Which situation represents the most appropriate trigger for calling `escalate_to_human`?

Customer Support Resolution Agent

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (`get_customer`, `lookup_order`, `process_refund`, `escalate_to_human`). Your target is 80%+ first-contact resolution while knowing when to escalate.

Production logs show the agent frequently calls `get_customer` when users ask about orders (e.g., "check my order #12345"), instead of calling `lookup_order`. Both tools have minimal descriptions ("Retrieves customer information" / "Retrieves order details") and accept similar identifier formats. What's the most effective first step to

Customer Support Resolution Agent

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (`get_customer`, `lookup_order`, `process_refund`, `escalate_to_human`). Your target is 80%+ first-contact resolution while knowing when to escalate.

61
62
63